

HELLO WORLD: p5.js

THEORY / TEXT / IMAGE / ERROR MESSAGE / PROMPT

A first p5.js reference you can actually use while coding.

THEORY	TEXT	IMAGE	ERROR	PROMPT
Write the rule that predicts the next change.	Run one complete sketch.js.	Compare the canvas with the expected output.	Read the console before guessing.	Ask for one complete sketch, not fragments.

START HERE

1. Open editor.p5js.org.
2. Put the whole sketch into sketch.js.
3. Press Play.
4. Compare the canvas to the IMAGE.
5. Make the listed mistake on purpose. Read the ERROR MESSAGE. Repair it.
6. Change one number and predict the result before you run it.

WHAT "THEORY" MEANS ON THESE PAGES

Not an invisible mental possession. Write a rule that can be tested: what runs, what changes, when it changes, and what must stay true. If the rule cannot predict a next frame or guide a repair, rewrite the rule.

The current p5.js beginner tutorials start from `setup()`, `draw()`, `createCanvas()`, and `background()`. The p5.js debugging guide teaches beginners to use the console and distinguishes syntax, reference, and type errors. [1-3]

1. HELLO CANVAS

Run the exact starter sketch that new p5.js projects begin with.

PROGRAM TEXT

```
function setup() {  
  createCanvas(400, 400);  
}  
  
function draw() {  
  background(220);  
}
```

IMAGE



THEORY = RULE THAT PREDICTS WHAT HAPPENS

setup() runs once. draw() runs repeatedly. createCanvas(400, 400) creates the drawing surface. background(220) repaints it gray. A still gray image does not tell you whether draw() ran once or thousands of times.

PROMPT TO MAKE THIS SKETCH

Write one complete p5.js sketch.js for a beginner. Use setup() and draw(). Create a 400 x 400 canvas and repaint it gray with background(220). Return only the complete sketch first, then explain setup and draw in two short sentences.

ERROR MESSAGE

SyntaxError: missing) after argument list

Break it: Delete the closing parenthesis in createCanvas(400, 400);.

Repair: Restore the missing). Read punctuation errors from the point the parser stops, not from where the canvas looks wrong.

[1] p5.js, "Setting Up Your Environment"; [2] p5.js, "Get Started." The p5.js Field Guide uses this SyntaxError as a beginner debugging example. [3]

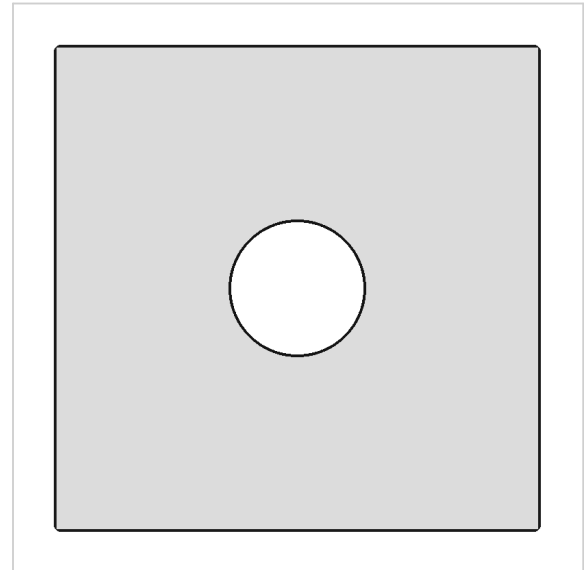
2. HELLO CIRCLE

Put one visible thing on the canvas.

PROGRAM TEXT

```
function setup() {  
  createCanvas(400, 400);  
}  
  
function draw() {  
  background(220);  
  circle(200, 200, 100);  
}
```

IMAGE



THEORY = RULE THAT PREDICTS WHAT HAPPENS

`circle(x, y, diameter)` uses the first two numbers for the center position and the third for size. On a 400 x 400 canvas, `circle(200, 200, 100)` places a 100-pixel circle in the center.

PROMPT TO MAKE THIS SKETCH

Write one complete p5.js sketch.js. Make a 400 x 400 gray canvas and draw one white circle with a black outline in the center. Use `circle(200, 200, 100)`. Keep the whole program under 10 lines.

ERROR MESSAGE

ReferenceError: myX is not defined

Break it: Change the circle line to `circle(myX, 200, 100);` without declaring `myX`.

Repair: Either put the number back, or declare `let myX = 200;` before `setup()`. A `ReferenceError` points to a name the program cannot currently resolve.

[1] The p5.js setup tutorial introduces drawing a circle after `background()`. [3] The Field Guide gives “`ReferenceError: myX is not defined`” as an example.

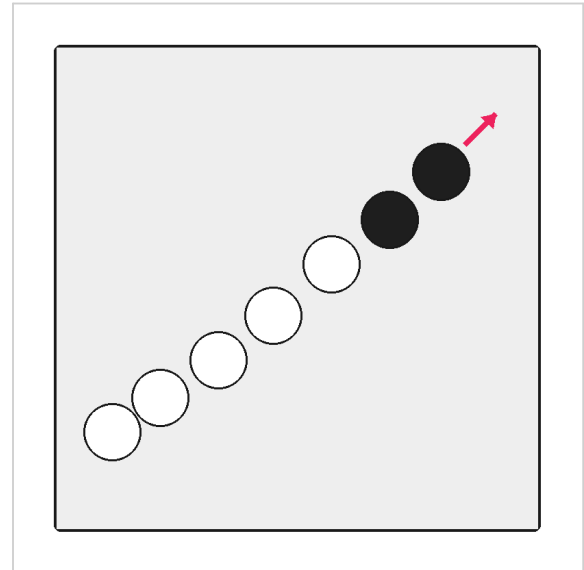
3. HELLO MOUSE

Make the sketch respond to a person immediately.

PROGRAM TEXT

```
function setup() {  
  createCanvas(400, 400);  
}  
  
function draw() {  
  if (mouseIsPressed) {  
    fill(0);  
  } else {  
    fill(255);  
  }  
  circle(mouseX, mouseY, 80);  
}
```

IMAGE



THEORY = RULE THAT PREDICTS WHAT HAPPENS

mouseX and mouseY supply the current pointer position. mouseIsPressed selects black or white. Because this sketch does not repaint the background inside draw(), old circles stay on the canvas and form a trail.

PROMPT TO MAKE THIS SKETCH

Write one complete p5.js sketch.js. Create a 400 x 400 canvas. Draw an 80-pixel circle at mouseX, mouseY. Make the circle white normally and black while the mouse button is pressed. Do not clear the background in draw(), so the circles leave a trail.

ERROR MESSAGE

ReferenceError: myX is not defined

Break it: Replace mouseX with myX but do not define myX.

Repair: Restore mouseX, or deliberately introduce and declare your own variable. Use the name in the error message as the search target.

[2] This follows the canonical first interactive sketch shown in p5.js "Get Started." [3] ReferenceError guidance comes from the p5.js Field Guide.

4. HELLO STATE

Make one value survive from one frame to the next.

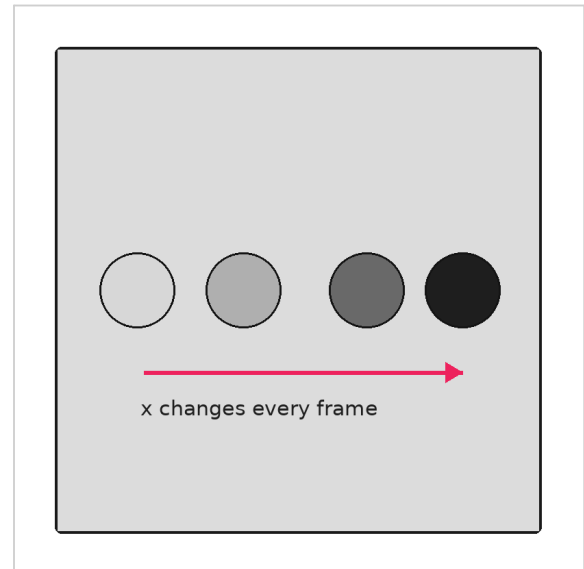
PROGRAM TEXT

```
let x = 50;

function setup() {
  createCanvas(400, 400);
}

function draw() {
  background(220);
  circle(x, 200, 80);
  x = x + 2;
}
```

IMAGE



THEORY = RULE THAT PREDICTS WHAT HAPPENS

At frame t , draw the circle at x . Then replace x with $x + 2$. The next frame uses the new value. The rule is $x(\text{next}) = x(\text{now}) + 2$. This is the smallest useful model of persistent state.

PROMPT TO MAKE THIS SKETCH

Write one complete p5.js sketch.js. Create a 400 x 400 gray canvas. Start a circle at $x = 50$, $y = 200$. Move it 2 pixels to the right every frame using one global variable named x . Keep the code simple enough for a first-day learner.

ERROR MESSAGE

p5.js says: "x" is not defined in the current scope.

Break it: Move `let x = 50;` inside `setup()` while still using `x` inside `draw()`.

Repair: Declare `x` outside both functions so both can access the same persistent variable. Scope errors tell you that the name exists in the wrong region or not at all.

[3] p5.js Friendly Error System documentation describes scope ReferenceErrors and advises checking scope, spelling, and case.

5. HELLO RULE

Make motion obey a boundary.

PROGRAM TEXT

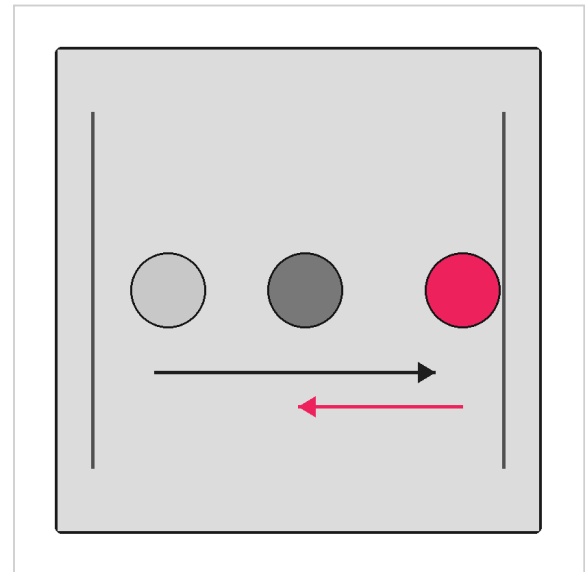
```
let x = 50;
let speed = 3;
let diameter = 80;

function setup() {
  createCanvas(400, 400);
}

function draw() {
  background(220);
  circle(x, 200, diameter);
  x = x + speed;

  if (x + diameter / 2 >= width ||
      x - diameter / 2 <= 0) {
    speed = speed * -1;
  }
}
```

IMAGE



THEORY = RULE THAT PREDICTS WHAT HAPPENS

The circle has a center x , a radius $\text{diameter} / 2$, and a signed speed. The rule is: if the right or left edge reaches the canvas boundary, reverse the sign of speed. The image shows a position; the rule predicts what happens next.

PROMPT TO MAKE THIS SKETCH

Write one complete p5.js sketch.js. Create a 400 x 400 canvas. Draw an 80-pixel circle at $y = 200$. Store x , speed , and diameter in variables. Move the circle horizontally. Reverse speed when either edge of the circle reaches the canvas edge. Use width rather than the literal number 400 in the collision test.

ERROR MESSAGE

NO ERROR MESSAGE - the sketch can be wrong and still run.

Break it: Change the right-edge test to `if (x >= width)`. The circle center will reach the edge only after half the circle has already left the canvas.

Repair: Compare the invariant to the quantities: $\text{right edge} = x + \text{diameter} / 2$. Repair the test, not the final pixels. Then change diameter to 120 to make sure the rule still works.

[3] The p5.js debugging guide emphasizes console logging and program flow. Naur's theory-building account makes modification a key test of program knowledge. [4]

6. HELLO BUG: WHEN THE CONSOLE SAYS NOTHING

The most important beginner bug is often a program that runs exactly as written - but not as intended.

WHOLE BUGGY SKETCH

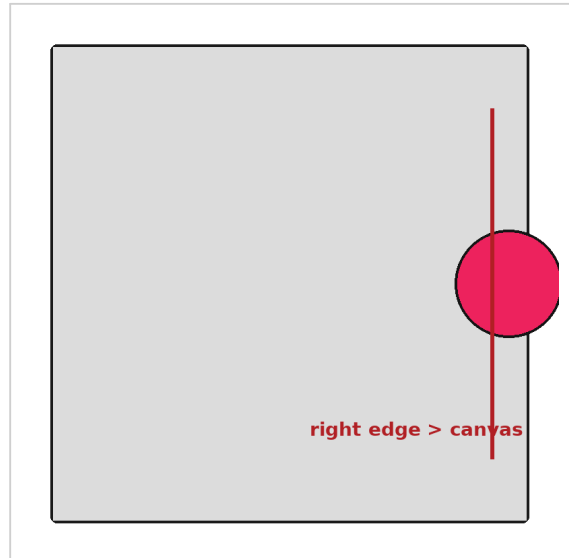
```
let x = 50;
let speed = 3;
let diameter = 80;

function setup() {
  createCanvas(400, 400);
}

function draw() {
  background(220);
  circle(x, 200, diameter);
  x = x + speed;

  // BUG: tests the center, not the right edge.
  if (x >= width) {
    speed = speed * -1;
  }
}
```

IMAGE



ERROR MESSAGE

NONE.

Syntax is valid. Names exist. Types are legal. The rule is simply wrong.

USE THE CONSOLE AS A SECOND CANVAS

Add this line inside `draw()` and watch the numbers:

```
console.log({ x, rightEdge: x + diameter / 2, width });
```

`x = 370` can still look legal while `rightEdge = 410` has already violated a 400-pixel boundary. Find the first violated rule, then repair the test.

NEGATIVE-SPACE DEBUGGING

When a repair attempt becomes cluttered, ask: What did not happen? Which condition did not fire? Which value was not preserved? Which fact do I not need? Goodin's NASA paper operationalizes negative space by asking what is not needed or not important during problem definition. [5]

[4] Peter Naur, "Programming as Theory Building" (1985). [5] Ronnie Goodin, NASA, "The Art World's Concept of Negative Space Applied to System Safety Management" (2005).

7. READ THE ERROR MESSAGE BEFORE YOU EDIT

Three beginner errors. Three different actions. These examples follow the p5.js Field Guide to Debugging. [3]

SyntaxError or	SyntaxError: missing) after argument list	Look for missing or misplaced punctuation:), },], comma, quote. Compare the whole function against a working reference.
Reference Error	ReferenceError: myX is not defined	Search for the exact name. Check declaration, spelling, capitalization, and scope.
Type / parameter error	p5.js says: circle() was expecting Number for the first parameter, received string instead.	Find the value passed into the named parameter. Check whether it is a number, string, object, or other type.
Logic bug	NO ERROR MESSAGE	State the rule the sketch should obey. Log the values used by that rule. Find the first frame where the rule becomes false.

THE 30-SECOND DEBUG LOOP

1. Read the first error line.
2. Go to the named line or variable.
3. Change one thing.
4. Run again.
5. If there is no error message, log the values that implement the rule.
6. Test a second case so the “repair” is not just a lucky patch.

p5.js also provides a Friendly Error System that supplements browser errors with beginner-oriented explanations and reference links. [6]

8. PROMPT A COMPLETE p5.js SKETCH - NOT A FRAGMENT

Use this when you ask an AI system to make or repair a first p5.js program.

Write ONE COMPLETE p5.js sketch.js for a beginner.

GOAL: [describe the visible behavior]

CANVAS: 400 x 400

LIBRARY: p5.js 2.x built-ins only

Requirements:

- Include the complete function setup() and function draw().
- Declare every variable the sketch uses.
- Keep the first version under 20 lines when possible.
- Do not return HTML, CSS, React, or external libraries.
- After the complete sketch, write exactly five short items:
THEORY: the rule that predicts what changes.
IMAGE: what should appear on the canvas.
ERROR: one realistic beginner mistake and its console message.
REPAIR: the smallest correction.
TEST: one number or condition to change to verify the repair.

If I paste an error message, diagnose that exact message before rewriting code.

EXAMPLE GOALS

Make the default gray p5.js canvas.

Draw a circle in the center.

Make a circle follow the mouse and turn black while pressed.

Move a circle 2 pixels right each frame.

Bounce a circle between the left and right edges.

ACCEPT THE ANSWER ONLY IF

You can paste the entire sketch into sketch.js; every variable is declared; the IMAGE description matches the canvas; the THEORY predicts a change; the ERROR names an observable failure; and the TEST can prove the repair still works after a modification.

9. ONE-PAGE REFERENCE

For any first p5.js sketch, fill these five boxes in order.

THEORY	What exact rule predicts the next frame or event?	$x(\text{next}) = x(\text{now}) + \text{speed}$
TEXT	What is the complete sketch.js that implements the rule?	setup() + draw() + declared variables
IMAGE	What should a person literally see?	gray 400 x 400 canvas; circle at x=200, y=200
ERROR	What does the console say - or does it say nothing?	ReferenceError: myX is not defined
PROMPT	What complete program are you asking another system to return?	one complete sketch.js + rule + expected image + bug + repair + test

THE TEST

Do not end with “it runs.” Change one number, one variable, or one condition. Predict what should happen. Run it. If the result differs, use the error message or console trace to locate the first broken rule.

SOURCES

[1] p5.js. “Setting Up Your Environment.” p5js.org/tutorials/setting-up-your-environment/. Beginner setup, starter code, circle and mouse examples.

[2] p5.js. “Get Started.” p5js.org/tutorials/get-started/. setup(), draw(), canvas lifecycle, shapes and interactivity.

[3] McIntyre, Nick, et al. “Field Guide to Debugging.” p5js.org/tutorials/field-guide-to-debugging/. SyntaxError, ReferenceError, type/parameter errors, console-based debugging.

[4] Naur, Peter. “Programming as Theory Building.” *Microprocessing and Microprogramming* 15(5), 1985, 253-261. DOI: 10.1016/0165-6074(85)90032-8.

[5] Goodin, James Ronald (Ronnie). “The Art World’s Concept of Negative Space Applied to System Safety Management.” NASA Kennedy Space Center, conference paper, 2005. NASA NTRS 20110024177.

[6] p5.js. “Friendly Error System (FES).” p5js.org/contribute/friendly_error_system/. Beginner-oriented console explanations and reference links.